# Package ‘FMLE’

March 10, 2026

**Type** Package

**Title** Fuzzy Mixture of Linear Experts for Single-Cell Protein Abundance Prediction

**Version** 0.1.0

**Description** Implements Fuzzy Mixture of Linear Experts (FMLE) for learning mappings from single-cell RNA expression to surface protein abundance, as measured for example in CITE-seq experiments. FMLE combines a fuzzy c-means gating network in a low-dimensional representation (e.g. PCA, embeddings or other latent spaces) with multiple linear experts operating on high-dimensional gene expression features. The model supports both single-protein (single-task) and multi-protein (multi-task) prediction, optional L1 penalization of expert coefficients (lasso plus post-ridge), and returns predictive means together with decomposed uncertainty (aleatoric and parameter). Although implemented in a general form for high-dimensional regression, the primary focus is accurate and interpretable prediction of protein abundance at single-cell resolution across donors, batches and antibody panels.

**License** GPL-3

**Encoding** UTF-8

**Roxygen** list(markdown = TRUE)

**RoxygenNote** 7.3.3

**Depends** R (>= 4.1.0)

**Imports** stats, glmnet, future.apply

**Suggests** tibble, ggplot2, testthat (>= 3.0.0), knitr, rmarkdown

**VignetteBuilder** knitr

**Config/testthat/edition** 3

**NeedsCompilation** no

**Author** Vikas Singh [aut, cre]

**Maintainer** Vikas Singh <vikkysingh07@gmail.com>

# Contents



---

`FMLE-package` *FMLE: Fuzzy Mixture of Linear Experts for Single-Cell Protein Prediction*

---

### Description

The FMLE package implements fuzzy mixture-of-experts models tailored to predicting surface protein abundance from single-cell RNA data, such as CITE-seq experiments. A fuzzy c-means gating network in a low-dimensional representation (e.g. principal components, latent embeddings, spatial coordinates) softly assigns each cell to multiple linear experts that act on high-dimensional gene expression features.

### Details

The package provides:

- `fmle_train` and `fmle_predict` for single-protein (single-task) regression.
- `fmle_train_mt` and `fmle_predict_mt` for multi-protein (multi-task) prediction with shared fuzzy gates and protein-specific experts.
- Optional L1 penalization for expert coefficients (lasso plus post-ridge), combined with a small ridge term for numerical stability.
- Prediction uncertainty with separate aleatoric and parameter components.

While FMLE can be applied to general high-dimensional regression problems, its primary motivation is accurate and interpretable prediction of protein abundance at single-cell resolution, across donors, batches and antibody panels.

### Author(s)

**Maintainer**: Vikas Singh <vikkysingh07@gmail.com>

---

`cap_and_scale_fit` *Cap and log-scale a response vector*

---

### Description

Cap and log-scale a response vector

### Usage

```
cap_and_scale_fit(y, q = 0.995, eps = 1e-08)
```

### Arguments

| | |
|---|---|
| `y` | numeric response (e.g., ADT counts) |
| `q` | upper quantile for capping (default 0.995) |
| `eps` | Small positive constant used for numerical stability. |

### Value

scaled numeric vector

---

`fcm_fit` *Fuzzy c-means clustering for gating*

---

### Description

Fuzzy c-means clustering for gating

### Usage

```
fcm_fit(Z, R, m = 1.8, max_iter = 200, tol = 1e-05, seed = 1, verbose = FALSE)
```

### Arguments

| | |
|---|---|
| `Z` | numeric matrix (cells × latent dimensions, e.g. PCs) |
| `R` | number of clusters / experts |
| `m` | fuzzifier (>1) |
| `max_iter` | maximum iterations |
| `tol` | convergence tolerance |
| `seed` | random seed |
| `verbose` | logical; print progress |

### Value

list with membership matrix U, centers, objective J, etc.

`fmle_cv_mt_parallel` *Multi-task cross-validation for FMLE*

## Description

Performs K-fold multi-task cross-validation over grids of R, m, and lambda_l1 using `fmle_train_mt` / `fmle_predict_mt`, with optional task weighting and grouped folds.

## Usage

```
fmle_cv_mt_parallel(
  X,
  Y,
  Z,
  R_grid = c(4, 6, 8),
  m_grid = c(1.6, 1.8, 2),
  lambda_grid = c(0, 0.001, 0.01, 0.03),
  folds = 5,
  groups = NULL,
  ridge = 1e-06,
  standardize = TRUE,
  seed = 1,
  exec = c("sequential", "future"),
  verbose = TRUE,
  task_weights = NULL
)
```

## Arguments

| | |
|---|---|
| `X` | matrix, n x p features. |
| `Y` | matrix, n x T responses (e.g., multiple proteins). |
| `Z` | matrix, n x d gating features. |
| `R_grid` | grid of R values. |
| `m_grid` | grid of m values. |
| `lambda_grid` | grid of lambda_l1 values. |
| `folds` | number of folds. |
| `groups` | optional factor of length n for grouped CV. |
| `ridge` | ridge penalty. |
| `standardize` | logical. |
| `seed` | random seed. |
| `exec` | "sequential" or "future". |
| `verbose` | logical. |
| `task_weights` | optional length-T vector of task weights; NULL = equal. |

### Value

A list with components:

**best** List with best (R, m, lambda, mse).

**table** Data frame of configs and their weighted mean MSE.

---

## `fmle_cv_parallel` *Cross-validation for single-task FMLE*

---

### Description

Performs K-fold cross-validation over grids of R (number of experts), m (fuzzifier), and lambda_l1 (L1 penalty), optionally using grouped folds and future-based parallelization.

### Usage

```
fmle_cv_parallel(
  X,
  y,
  Z,
  R_grid = c(4, 6, 8),
  m_grid = c(1.6, 1.8, 2),
  lambda_grid = c(0, 0.001, 0.01, 0.03),
  folds = 5,
  groups = NULL,
  ridge = 1e-06,
  standardize = TRUE,
  seed = 1,
  exec = c("sequential", "future"),
  verbose = TRUE,
  q = 0.995
)
```

### Arguments

| | |
|---|---|
| `X` | matrix, n x p features. |
| `y` | numeric response vector of length n. |
| `Z` | matrix, n x d gating features (e.g., PCs). |
| `R_grid` | grid of R values. |
| `m_grid` | grid of m values. |
| `lambda_grid` | grid of lambda_l1 values. |
| `folds` | number of CV folds. |
| `groups` | optional factor of length n for grouped CV; NULL gives random folds. |
| `ridge` | small ridge penalty for stability |

| | |
|---|---|
| standardize | logical; standardize X and Z |
| seed | random seed |
| exec | character, "sequential" or "future" (for future_lapply). |
| verbose | logical; print progress messages. |
| q | Quantile used to cap the response before log1p/scale preprocessing in cross-validation. |

## Value

A list with components:

**best** List with the best (R, m, lambda, mse).

**table** Data frame of all configs and their CV MSE.

---

`fmle_predict` *Predict from an FMLE model on single task*

---

## Description

Predict from an FMLE model on single task

## Usage

```
fmle_predict(model, X_new, Z_new = NULL, return_se = TRUE)
```

## Arguments

| | |
|---|---|
| model | object of class "fmle" |
| X_new | new design matrix (cells x p) |
| Z_new | new gating features (cells x d); if NULL, uses X_new |
| return_se | logical; if TRUE, also return uncertainty estimates |

## Value

list with components `mean`, `alpha`, `mu_r`, and optionally `var`, `se`

## fmle_predict_mt *Predict from a multi-task FMLE model*

### Description

Predict from a multi-task FMLE model

### Usage

```
fmle_predict_mt(model, X_new, Z_new = NULL, return_se = TRUE)
```

### Arguments

| | |
|---|---|
| `model` | object of class "fmle_mt" |
| `X_new` | new feature matrix |
| `Z_new` | new gating feature matrix |
| `return_se` | logical |

### Value

list with `mean` (n x T), `alpha`, and optional `var`, `se`

## fmle_train *Train FMLE model for a single task*

### Description

Train FMLE model for a single task

### Usage

```
fmle_train(
  X,
  y,
  Z = NULL,
  R = 6,
  m = 1.8,
  lambda_l1 = 0,
  ridge = 1e-06,
  standardize = TRUE,
  fcm_max_iter = 200,
  fcm_tol = 1e-05,
  seed = 1,
  verbose = FALSE
)
```

### Arguments

| | |
|---|---|
| `X` | matrix, n x p features (e.g., HVG expression) |
| `y` | numeric vector length n (response) |
| `Z` | matrix, n x d gating features (e.g., PCs). If NULL, uses X. |
| `R` | number of experts |
| `m` | fuzzifier (>1) |
| `lambda_l1` | L1 penalty (scalar or length-R vector) |
| `ridge` | small ridge penalty for stability |
| `standardize` | logical; standardize X and Z |
| `fcm_max_iter` | max FCM iterations |
| `fcm_tol` | tolerance for FCM convergence |
| `seed` | random seed |
| `verbose` | logical |

### Value

object of class "`fmle`"

---

`fmle_train_mt` *Train FMLE model for a multi-task*

---

### Description

Train FMLE model for a multi-task

### Usage

```
fmle_train_mt(
  X,
  Y,
  Z = NULL,
  R = 6,
  m = 1.8,
  lambda_l1 = 0,
  ridge = 1e-06,
  standardize = TRUE,
  fcm_max_iter = 200,
  fcm_tol = 1e-05,
  seed = 1,
  verbose = FALSE
)
```

## Arguments

| | |
|---|---|
| X | matrix, n x p features |
| Y | matrix, n x T responses (T tasks, e.g., multiple proteins) |
| Z | matrix, n x d gating features |
| R | number of experts |
| m | fuzzifier |
| lambda_l1 | penalty (scalar, length-R, length-T, RxT matrix, or list) |
| ridge | ridge penalty |
| standardize | logical |
| fcm_max_iter | max FCM iterations |
| fcm_tol | tolerance |
| seed | random seed |
| verbose | logical |

## Value

object of class "fmle_mt"

# Index